

JAVA INTERVIEW PREPARATION

CHAPTER 13

PATTERN MATCHING AND MODERN SWITCH

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Pattern Matching and Modern switch: Type-Safe Branching without Casting Noise

Senior / Lead Java Developer Reading Chapter

Modern Java has gradually expanded pattern matching and switch. Switch expressions became permanent in Java 14, pattern matching for `instanceof` in Java 16, and record patterns plus pattern matching for switch in Java 21. Together they let code test a value's shape, bind useful variables, and produce a result with less casting and less accidental fall-through.

The syntax is concise, but senior-level reasoning is about semantics: variable scope, exhaustiveness, dominance, null handling, guards, hierarchy evolution, and the boundary between useful data-oriented processing and procedural type switching.

Switch Statements versus Switch Expressions

Traditional switch is a statement with colon labels and possible fall-through:

```
String description;

switch (status) {
    case NEW:
        description = "New";
        break;
    case PROCESSING:
        description = "Processing";
        break;
    default:
        description = "Complete";
}
```

A switch expression returns a value:

```
String description = switch (status) {
    case NEW -> "New";
    case PROCESSING -> "Processing";
    case COMPLETED, CANCELLED -> "Complete";
};
```

Arrow rules do not fall through. Multiple labels can share one result.

```
selector
|
+-- NEW -----> "New"
+-- PROCESSING -----> "Processing"
+-- COMPLETED/CANCELLED -> "Complete"
|
exactly one rule produces the expression result
```

Switch expressions encourage total value initialization and eliminate the partially assigned local variable. Traditional colon-style groups remain supported when fall-through is deliberately needed, but most business mapping code is clearer with arrow rules.

Block Rules and yield

An arrow rule can use a block when calculation requires several statements. The block returns a switch value with `yield`.

```
Money fee = switch (customer.tier()) {
    case STANDARD -> baseFee;
    case PREMIUM -> Money.zero(currency);
    case PARTNER -> {
        Money negotiated = partnerPricing.lookup(customer.id());
        audit.recordPartnerFee(customer.id(), negotiated);
        yield negotiated;
    }
};
```

`yield` returns from the switch expression, not from the enclosing method. Do not confuse it with `return`.

A switch expression must produce a compatible result on every normal path. Throwing is also a valid rule outcome:

```
case SUSPENDED -> throw new SuspendedCustomerException(customer.id());
```

Keep blocks focused. If a case contains an entire workflow, extract a named method so the switch remains a readable decision table.

Exhaustiveness

A switch expression must be exhaustive. Enum switches can list every current constant:

```
int priority = switch (severity) {
    case LOW -> 1;
    case MEDIUM -> 2;
    case HIGH -> 3;
    case CRITICAL -> 4;
};
```

No default is required because every enum constant is covered. Adding a new constant causes source code to fail compilation when recompiled, identifying the mapping that needs a decision.

For sealed hierarchies, all permitted cases can provide the same benefit:

```
String code(PaymentResult result) {
    return switch (result) {
        case Approved approved -> "APPROVED";
        case Declined declined -> "DECLINED";
        case Pending pending -> "PENDING";
    };
}
```

Avoid a default solely for convenience when compile-time evolution pressure is valuable. A default can silently absorb a newly added status or subtype.

At runtime, separate compilation can create a compatibility gap: code compiled against an older enum or sealed hierarchy may encounter a newer constant or implementation. Exhaustiveness helps source evolution but does not replace deployment and protocol compatibility planning.

Pattern Matching for instanceof

Before pattern matching, type testing and casting were separate:

```
if (value instanceof Customer) {
    Customer customer = (Customer) value;
    return customer.displayName();
}
```

A type pattern combines the test and binding:

```
if (value instanceof Customer customer) {
    return customer.displayName();
}
```

The pattern variable exists only where the compiler knows the match succeeded. This is flow scoping.

```
if (!(value instanceof Customer customer)) {
    throw new IllegalArgumentException("customer required");
}

return customer.displayName(); // match must have succeeded here
```

The compiler analyzes control flow, not only braces. Because the failed branch always throws, `customer` is in scope afterward.

Flow Scoping with Boolean Expressions

Short-circuit operators affect pattern-variable scope:

```
if (value instanceof String text && !text.isBlank()) {
    use(text);
}
```

The right side of `&&` runs only when the pattern matched, so `text` is available.

This does not compile:

```
// text is not guaranteed to exist on the right side
// if (value instanceof String text || text.isBlank()) { }
```

If the left side is false, `||` evaluates the right side even though no String was bound.

Pattern variables are not final, but reassigning them usually harms clarity. Treat them as the value proven by the match and introduce another variable for transformations.

Type Patterns in switch

Java 21 permits type patterns directly in switch:

```
BigDecimal amount(Event event) {
    return switch (event) {
        case OrderPlaced placed -> placed.total();
        case RefundIssued refund -> refund.amount().negate();
        case AccountOpened ignored -> BigDecimal.ZERO;
    };
}
```

Each rule both tests the runtime type and binds a correctly typed variable. This is especially useful with sealed interfaces whose alternatives represent data cases.

For an open hierarchy, a catch-all rule remains necessary:

```
String describe(Object value) {
    return switch (value) {
        case String text -> "text: " + text;
        case Number number -> "number: " + number;
        default -> "other: " + value;
    };
}
```

Do not use a large type switch over an open polymorphic service hierarchy when virtual dispatch belongs on the types. Pattern switch is strongest when consumers intentionally interpret a data-oriented family.

Dominance and Case Ordering

Pattern cases are considered in order, and a broader earlier pattern can dominate a narrower later one.

```
switch (value) {
    case CharSequence sequence -> use(sequence);
    // case String text -> use(text); // dominated and unreachable
    default -> useUnknown(value);
}
```

Every String is already a CharSequence, so the String case can never run. The compiler rejects it.

Order from specific to general:

```
switch (value) {
    case String text -> useText(text);
    case CharSequence sequence -> useSequence(sequence);
    default -> useUnknown(value);
}
```

Dominance is a correctness feature. It prevents refactoring from leaving silently unreachable business rules.

Guards with when

A guarded pattern adds a boolean condition after a successful match:

```
String category(Transaction transaction) {
    return switch (transaction) {
        case CardPayment payment when payment.amount().isGreaterThan(highValue) ->
            "HIGH_VALUE_CARD";
        case CardPayment payment -> "CARD";
        case BankTransfer transfer -> "TRANSFER";
    };
}
```

The guarded high-value case must appear before the unguarded CardPayment case, because the unguarded type pattern accepts every CardPayment.

Guards should be predicates over already available data. Hidden database calls, remote calls, or mutations inside guards make case selection expensive and surprising. Prepare required context before switching or call a clearly named method.

Several guarded rules for one type can indicate a decision table that deserves a domain policy object. Pattern matching improves syntax; it does not replace decomposition.

Null Handling

Historically, switching on null threw `NullPointerException`. Pattern switches can handle null explicitly:

```
String format(Object value) {
    return switch (value) {
        case null -> "<missing>";
        case String text -> text;
        default -> value.toString();
    };
}
```

Without `case null`, a null selector still throws before matching other cases. A `default` does not automatically match null.

Handling null explicitly is not always desirable. If null violates the method contract, allow or deliberately throw a clear exception at the boundary rather than turning it into a normal switch case.

```
Objects.requireNonNull(value, "value");
return switch (value) { ... };
```

Null policy should come from the domain, not from the convenience of a case label.

Record Patterns

Java 21 record patterns test a record type and deconstruct its components:

```
String label(Object value) {
    return switch (value) {
        case CustomerSummary(CustomerId id, String name) ->
            id + ": " + name;
        default -> "unknown";
    };
}
```

Instead of binding the whole record and calling its accessors, the pattern binds component values directly.

Record patterns can nest:

```
record Address(String city, Country country) {}
record Customer(String name, Address address) {}

String destination(Object value) {
    return switch (value) {
        case Customer(String name,
            Address(String city, Country country)) ->
            name + " in " + city + ", " + country.code();
        default -> "unknown";
    };
}
```

```
Customer
+-- name -----> name
+-- Address
    +-- city -----> city
    +-- Country -----> country
```

Nested patterns are effective when the shape itself is the rule. If deconstruction becomes wide or deeply nested, bind the larger object and call a named method. Readability is more important than demonstrating maximum pattern depth.

Record Pattern Safety and Accessors

A record pattern uses the record's component accessors conceptually to obtain values. If an accessor has surprising behavior or throws, deconstruction inherits that behavior. Records should keep accessors transparent and perform normalization during construction.

Nested component patterns do not magically accept null. If a nested record component is null, a record pattern requiring that nested type does not match. A broader case may handle the containing record, or the record invariant should reject null at construction.

```
case Customer(String name, null) -> name + " has no address";
case Customer(String name, Address(String city, Country country)) -> ...
```

Prefer domain invariants over proliferating null patterns when absence is not legitimate.

Generic Types and Erasure

Pattern matching follows Java's reifiability rules. Because generic type arguments are erased, code cannot generally test `value instanceof List<String>`.

```
if (value instanceof List<?> values) {
    // element type is still unknown
}
```

A pattern can establish that the value is a List, but each element still requires its own type contract or validation. Do not cast a `List<?>` to `List<String>` based on a few inspected elements; heap pollution may surface later.

Sealed generic hierarchies can sometimes provide more type information through their permitted cases, but the compiler remains constrained by erasure and applicable type arguments. Understand the actual static selector type rather than assuming runtime generics are available.

Variables, Scope, and Naming

Pattern variable names should reflect the case meaning:

```
case PaymentApproved approved -> approved.transactionId();
case PaymentDeclined declined -> declined.reason();
```

The variable belongs only to its rule or flow-proven scope. Different rules may reuse a name because their scopes do not overlap.

Avoid names such as `obj`, `x`, or repeated `value` in a large switch. Pattern matching removes cast noise, so names become the primary explanation of each branch.

Side Effects and Control Flow

A switch expression is easiest to reason about when each rule calculates a value. Side effects may be necessary, but a switch that sends messages, updates databases, changes metrics, and returns results from each case can hide transaction and partial-failure behavior.

```

Handler handler = switch (command) {
    case CreateOrder create -> createOrderHandler;
    case CancelOrder cancel -> cancelOrderHandler;
    case ShipOrder ship -> shipOrderHandler;
};

return handler.handle(command);

```

This example may itself need generic redesign, but it demonstrates separating selection from execution. In many cases, polymorphic `command.executeWith(handler)` or a dispatcher registry is clearer.

Use pattern switch as a decision structure, not as a container for unrelated workflows.

Refactoring and Evolution

Pattern matching makes variant use sites explicit. This is excellent for a closed internal model: adding a sealed subtype reveals every exhaustive consumer during compilation.

It can also create a shotgun-change problem when dozens of services switch on the same hierarchy for similar behavior. If each new case requires the same edit everywhere, consider moving stable behavior onto the variant, centralizing a mapper, or using a visitor-like abstraction.

Conversely, if every new operation requires modifying every variant class, external pattern processing may be more maintainable. The classic expression problem remains a tradeoff between adding variants and adding operations.

frequent new variants -> polymorphism can localize change
 frequent new operations -> external exhaustive switches can localize change

Testing Modern switch and Patterns

Test each meaningful rule, guard boundary, and null policy.

```

@ParameterizedTest
@MethodSource("transactions")
void everyTransactionHasCategory(Transaction transaction) {
    assertFalse(category(transaction).isBlank());
}

```

Important tests include:

- Values exactly at guard thresholds.
- Specific patterns before broader patterns.
- Null selectors when null is supported or forbidden.
- Every enum constant and permitted sealed case.
- Record components that may be absent.
- Serialization creating each runtime variant.
- Behavior after adding a new domain case.

Compiler checking handles syntactic dominance and source exhaustiveness, but it cannot prove that `HIGH_VALUE_CARD` uses the correct threshold or that cases map to the correct HTTP status.

Production Perspective

Pattern-related incidents often come from defaults, hidden effects, and version drift. A default maps a new fraud case to "unknown" instead of blocking payment. A guard performs a repository call for every case evaluation. Null begins arriving after an integration change and causes NPE before default. A deeply nested record pattern assumes a component invariant that deserialization does not enforce. An old service encounters an enum constant introduced by a separately deployed producer.

When diagnosing or reviewing pattern code, ask:

- Is the selector family open or deliberately closed?
- Is the switch exhaustive, and is a default hiding evolution?
- Are cases ordered from specific to general?
- Are guards pure, cheap, and clearly named?
- What is the explicit null policy?
- Do record invariants make nested deconstruction safe?
- Would behavior belong more naturally on the subtype?
- How do independently deployed versions handle new variants?
- Does each rule calculate a value or hide an operational workflow?

Interview-Ready Summary

Switch expressions became permanent in Java 14 and use arrow rules to avoid accidental fall-through. They produce a value and must be exhaustive. A block rule returns its value with `yield`, while a rule may also throw.

Pattern matching for `instanceof`, permanent in Java 16, combines type testing and variable binding. Pattern variables use flow scoping, so they exist only where successful matching is proven. Short-circuit boolean operators determine where that proof holds.

Java 21 pattern switch supports type patterns, guards with `when`, explicit null cases, and exhaustive processing of sealed hierarchies. Broader patterns dominate narrower later patterns, so cases must be ordered from specific to general.

Java 21 record patterns deconstruct transparent record components and may nest. They improve code when data shape is the decision, but deep patterns reduce readability and depend on trustworthy record invariants. Generic pattern matching remains limited by type erasure.

Avoid default when compiler pressure to handle new enum or sealed cases is desirable. Use pattern processing for stable data-oriented families and multiple external operations; retain polymorphism when behavior belongs to extensible subtypes. Source exhaustiveness does not solve independent deployment or wire compatibility.

Revision Notes

- Switch expressions became permanent in Java 14.
- Arrow rules do not fall through.
- A switch expression produces a value and must be exhaustive.
- Use `yield` to return a value from a block rule.
- A rule may throw instead of yielding a normal value.
- Extract large rule blocks into named methods.
- Exhaustive enum switches reveal new constants after recompilation.

- Exhaustive sealed switches reveal new permitted subtypes.
- Avoid default when handling every new case is required.
- Default does not automatically match null in a pattern switch.
- Pattern matching for instanceof became permanent in Java 16.
- A type pattern combines a type test and a bound variable.
- Pattern variables use compiler-verified flow scoping.
- A variable is available on the right side of && after a successful pattern.
- It is not safely available on the right side of || after a possibly failed pattern.
- Pattern matching for switch became permanent in Java 21.
- Type switch is strongest with sealed, data-oriented families.
- Open hierarchies generally require a catch-all rule.
- Broader patterns dominate narrower patterns placed after them.
- Order type cases from specific to general.
- Guards use when and should be pure, cheap predicates.
- Put guarded cases before an unguarded case of the same type.
- Handle null explicitly only when null is a legitimate domain state.
- Otherwise validate null at the method boundary.
- Record patterns became permanent in Java 21.
- Record patterns test and deconstruct component values.
- Nested patterns are useful when shape itself drives the rule.
- Deep or wide patterns should be replaced with named methods for readability.
- Record patterns depend on transparent accessors and valid components.
- A nested type pattern does not match a null component.
- Type erasure prevents general instanceof checks such as List<String>.
- List<?> establishes the container type, not its element type.
- Pattern names should explain case meaning.
- Keep operational workflows out of large switch rules.
- Frequent new variants may favor polymorphism.
- Frequent new operations over stable variants may favor exhaustive switches.
- Source exhaustiveness does not guarantee wire or deployment compatibility.
- Test guard boundaries, null policy, every case, and business mapping.
- Compiler dominance checks do not prove business rules are correct.
- Use modern switch to make decisions explicit, not merely to shorten syntax.